

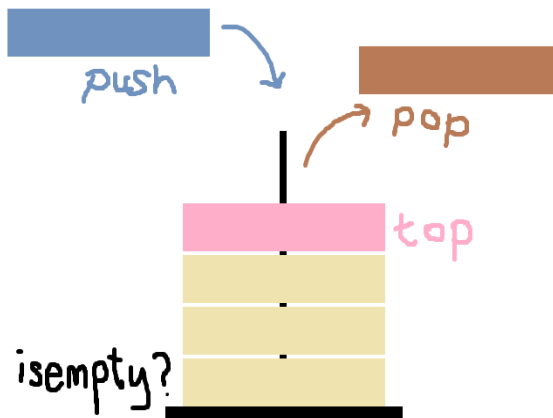
MODUL 10

(STACKS & QUEUES)

DESKRIPSI TEMA

Stack (tumpukan) adalah sebuah struktur data yang meletakkan elemen baru di atas elemen lainnya. Struktur data ini menggunakan konsep LIFO (Last In First Out). Dengan demikian, elemen terakhir yang dimasukkan ke dalam stack, merupakan elemen pertama yang akan diambil. Secara umum, operasi dalam struktur data stack terdiri dari 4 operasi, yaitu:

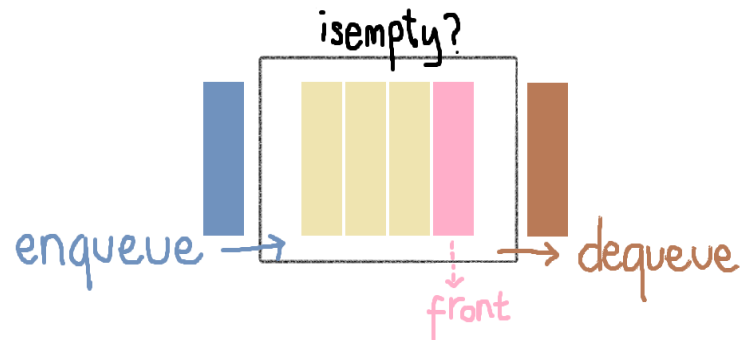
1. push : Memasukan data kedalam stack.
2. pop : Mengeluarkan data dari dalam stack.
3. isempty : Mengecek apakah stack memiliki isi/tidak.
return *TRUE/FALSE*
4. top : Mengakses elemen paling atas dari sebuah stack.
return *data*.



Queue (antrian) adalah sebuah struktur data yang menerapkan konsep FIFO (First In First Out). Dengan demikian, elemen yang pertama kali masuk ke dalam queue merupakan elemen yang pertama kali akan keluar dari dalam queue. Operasi dasar dalam struktur data ini secara umum dibagi menjadi:

1. enqueue : Memasukan data kedalam queue
2. dequeue : Mengeluarkan data dari dalam queue
3. isempty : Mengecek apakah queue memiliki isi/tidak.
return *TRUE/FALSE*
4. front : Mengakses elemen paling depan dari sebuah queue.

return data



CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan konsep stacks dan queues (C3).

PENUNJANG PRAKTIKUM

1. Software CodeBlocks

LANGKAH-LANGKAH PRAKTIKUM

Implementasi Stack

```
#include <stdio.h>
#include <malloc.h>

typedef struct data{
    int n;
    struct data *next;
}DATA;

void push(DATA **stack, int angka);
void pop(DATA **stack);
int isempty(DATA *stack);
int top(DATA *stack);

int main(int argc, char const *argv[]){
    DATA *stack;
    int n, angka;

    //PUSH
    stack = NULL;
    printf("Masukkan jumlah data: ");
    scanf("%d", &n); getchar();

    int i;
    for(i=0; i<n; i++){
        printf("Masukkan data ke-%d: ", i+1);
        scanf("%d", &angka); getchar();
        push(&stack, angka);
    }

    //POP
    while(!isempty(stack)){
        printf("%d ", top(stack));
        pop(&stack);
    }

    return 0;
}
```

```

void push(DATA **stack, int angka){
    DATA *node = (DATA*) malloc(sizeof(DATA));
    node->n = angka;
    node->next = NULL;

    if(!isempty(*stack)) node->next = *stack;
    *stack = node;
}

void pop(DATA **stack){
    DATA *hapus = *stack;
    *stack = (*stack)->next;
    free(hapus);
}

int isempty(DATA *stack){
    if(stack == NULL) return 1;
    else return 0;
}

int top(DATA *stack){
    return stack->n; //STACK !ISEMPTY
}

```

Tutorial 1 – Stack

1. Jika program sebelumnya dimodifikasi dan diberi input sebagai berikut: -1 2 4 -1 3 7 7 -1 0

apakah output dari program ini?

```

int main(int argc, char const *argv[]) {
    DATA *stack;
    int number;

    while(1) {
        scanf("%d", &number);

        if(number == 0) {
            break;
        }
        else if(number < 0) {
            if(!isempty(stack)) {
                pop(&stack);
            }
        }
        else {
            push(&stack, number);
        }
    }
}

```

2. Simpan pada file dengan nama **Week[xx]_[nim]_[nama]_01.txt**
e.g.: **Week10_10101_EzraAbednego_01.txt**

```
#include <stdio.h>
#include <malloc.h>

typedef struct data{
    int n;
    struct data *next;
}DATA;

void enqueue(DATA **head, DATA **tail, int angka);
void dequeue(DATA **head);
int isempty(DATA *head);
int front(DATA *head);

int main(int argc, char const *argv[]){
    DATA *head, *tail;
    int n, angka;

    //Enqueue
    head = tail = NULL;
    printf("Masukkan jumlah data: ");
    scanf("%d", &n); getchar();

    int i;
    for(i=0; i<n; i++){
        printf("Masukkan data ke-%d: ", i+1);
        scanf("%d", &angka); getchar();
        enqueue(&head, &tail, angka);
    }

    //Dequeue
    while(!isempty(head)){
        printf("%d ", front(head));
        dequeue(&head);
    }

    return 0;
}
```

```
void enqueue(DATA **head, DATA **tail, int angka){
    DATA *node = (DATA*) malloc(sizeof(DATA));
    node->n = angka;
    node->next = NULL;

    if(*head == NULL) *head = node;
    else (*tail)->next = node;

    *tail = node;
}

void dequeue(DATA **head){
    DATA *hapus = *head;
    *head = (*head)->next;
    free(hapus);
}

int isempty(DATA *head){
    if(head == NULL) return 1;
    else return 0;
}

int front(DATA *head){
    return head->n; //HEAD !ISEMPTY
}
```

Tutorial 2 – Queue

1. Jika program di atas dimodifikasi dan diberi input sebagai berikut: -1 2 4 -1 3 7 7 -1 0

apakah output dari program ini?

```
int main(int argc, char const *argv[]) {
    DATA *Head = NULL;
    int number;

    while(1) {
        scanf("%d", &number);

        if(number == 0) {
            break;
        }
        else if(number < 0) {
            if(!isempty(Head)) {
                dequeue(&Head);
            }
        }
        else {
            enqueue(&Head, number);
        }
    }
}
```

2. Simpan pada file dengan nama **Week[xx]_[nim]_[nama]_02.txt**
e.g.: **Week10_10101_LionelReinhartHalim_02.txt**

Challenge 1 – Stack

Salah satu implementasi dari struktur data stack adalah *Parentheses Checking*. Buatlah program yang akan memeriksa apakah input user merupakan *valid parenthesis expression*. Jenis kurung yang digunakan adalah: '{}()[]'. Program akan berhenti jika di input -1.

Untuk lebih jelasnya silahkan lihat gambar dibawah

```
Masukan pola: {}()[]  
Valid parenthesis expression  
  
Masukan pola: { ( ) [ ( ) ] }  
Valid parenthesis expression  
  
Masukan pola: ( [ ] )  
Invalid parenthesis expression  
  
Masukan pola: ( ( ) ) )  
Invalid parenthesis expression  
  
Masukan pola: a  
Invalid parenthesis expression  
  
Masukan pola: 2  
Invalid parenthesis expression  
  
Masukan pola: -  
Invalid parenthesis expression  
  
Masukan pola: _
```

Kumpulkan dengan nama file

Week[xx]_[nim]_[nama]_challengeStack.c

e.g.: **Week10_10101_BudimanPutraJaya_challengeStack.c**

Challenge 2 – Queue

Buatlah sebuah program queue yang menerima input angka secara terus menerus dengan ketentuan:

1. Jika queue kosong, input angka dan enqueue angka tersebut sebanyak angka tersebut.
2. Jika queue tidak kosong, saat input angka, lakukan pengecekan, bila angka yang di input lebih kecil dari front, maka lakukan dequeue sebanyak inputan tersebut (jangan lupa error handling ketika queue kosong saat sedang melakukan dequeue), bila angka yang di input lebih besar atau sama dengan front, lakukan enqueue sebanyak front tersebut.
3. Kumpulkan dengan nama file:

Week[xx]_[nim]_[nama]_challengeQueue.c

e.g.: **Week10_10101_BudimanPutraJaya_challengeQueue.c**

REFERENSI